COMP 018: Database Administration - Activity 4 Progress Report
Access Control, Database Users, Roles, Privileges, and Security Governance

Project Identification & Metadata

Course	: COMP 018 - Database Administration
Evaluating Group	: Group 4: Olivarez, Abangan, Pocsidio, Infantado, and Bartolata
Evaluating Group Repository	: https://github.com/Nycri/brewpos
Target Project Monitored	: HabalGo: Ride and Booking Management System
Target Project Group	: Group 6 (Royo, Villareal, Barte, Balansay, Remoquillo, Gumahin)
Target Project Repository	: https://github.com/jeanaroyo/HabalGo
Primary Database	: grp00_habalgodb (PostgreSQL 18.6 on x86_64 Windows, compiled by MSVC)
Target Schema	: habalgo
Restored Test Database	: grp00_habalgodb_restore_test

ACTIVITY 4: Database Users, Roles, Privileges, and Access Control

1.1 Access-Control Plan & Architecture

In the default configuration of Group 6's HabalGo application, the backend connected using the unrestricted administrative superuser account. Operating a public-facing ride-hailing and booking web application under a superuser account directly violates the Principle of Least Privilege, as any application flaw could allow an attacker to drop tables, purge transaction logs, or access unrelated databases on the host system.

An inspection of Group 6's initial access control script revealed three critical vulnerabilities: cluster-wide role namespace collisions resulting from generic role names shared across different groups, blanket record deletion permissions across all operational tables that threatened financial and trip audit logs, and the complete omission of a dedicated backup operator role.

To address these vulnerabilities, a role-based access control architecture was developed for HabalGo. This architecture introduced project-namespaced non-login group roles to prevent cross-project collisions, dedicated login accounts assigned to their respective group roles, and strictly scoped deletion privileges. The runtime application account was granted permissions to read, insert, and update operational tables while confining record deletion exclusively to driver-vehicle assignments. Read-only auditor accounts were granted querying access across operational data without modification privileges, and a dedicated backup role was established for database archiving without write capabilities. In addition, default connection and creation privileges were revoked from public roles across the database and schema boundaries.

1.2 Setup Execution & Access-Control Hierarchy

The access-control configuration script detached conflicting legacy roles and established three

non-login group roles representing runtime, read-only audit, and backup profiles. Matching login accounts were created with dedicated credentials and assigned to their respective group roles. Access privileges were revoked from public roles, and explicit database connection and schema usage permissions were assigned. Specific table grants were applied to each role according to operational requirements, and default schema privileges were configured to maintain these definitions for future database objects. System catalog queries confirmed that login accounts and group memberships accurately reflect the defined access control structure.

1.3 Verification Results: Runtime Account Testing

Verification testing demonstrated that the runtime account successfully reads active booking records, inserts new booking entries while advancing primary sequence counters, updates booking statuses, and unassigns vehicles within authorized tables. When deletion operations were attempted on booking, payment, or customer records, the database engine denied execution under the narrow deletion policy. Data definition operations, including table drop commands, were rejected due to missing object ownership permissions.

1.4 Verification Results: Read-Only Auditor Account Testing

Inspection testing confirmed that the read-only auditor account successfully executes multi-table join queries across bookings, customers, drivers, and payment records to monitor system activity. Attempted modification operations, including record insertions into location tables, payment status updates, row deletions on bookings, and table truncation commands, were uniformly blocked by the database engine.

1.5 Verification Results: Cross-Database & Administrative Boundary Testing

Administrative isolation testing demonstrated that database creation commands attempted under application accounts were denied due to missing creation privileges. Connection attempts to unrelated databases on the server instance failed because public connection permissions were revoked, restricting access strictly to authorized targets.

1.6 Issues, Fixes, and Feedback to Owners

During deployment evaluation, executing Group 6's original script triggered role collision errors due to generic group role names already present on the cluster. This was resolved by re-architecting role templates with project-namespaced prefixes. Seed script execution errors caused by duplicate primary key inserts were fixed by adding conflict resolution handling and resynchronizing identity sequences. Database name references were standardized across configuration files.

Feedback provided to Group 6 commended the well-modeled relational schema and strict constraint enforcement within the database. Recommendations advised prefixing group roles with project identifiers to prevent deployment failures on shared clusters, replacing blanket deletion grants with narrow table-level permissions to protect transactional history, and adding conflict handling in seed scripts to ensure idempotent execution.

CLASS GOOGLE SHEETS TRACKER ENTRY (Activity 4)

Column Header	Value
Project	Group 6 - HabalGo: Ride and Booking Management System
Repository cloned	https://github.com/jeanaroyo/HabalGo, Branch: main
Database setup result	Success (Initialized database grp00_habalgodb, created schema habalgo, loaded 8 tables, sequences, and operational seed data).
Access-control folder link	https://github.com/Nycri/brewpos/tree/main/access-contro
Access-control status	Completed
Feedback sent to owners	(05/11/2026)
PDF progress document	Activity4_Progress_Report_Group6.pdf
Last updated	2026-11-05

AI USAGE DECLARATION

Google Gemini 3.8 Flash was utilized during project evaluation to inspect database schema definitions, evaluate relational hierarchy structures, formulate role verification test scenarios, analyze database maintenance operations and query performance execution plans, verify backup creation and restoration procedures, and structure technical report documentation. All database statements, configuration checks, and verification procedures were directly executed and validated against PostgreSQL 18.6 within the local database environment.